

ASSIGNMENT FINAL REPORT

Qualification	Pearson BTEC Level 5 Higher National Diploma in Computing		
Unit number and title	Unit 20: Applied Programming and Design Principles		
Submission date	26/6/2026	Date Received 1 st Submission	
Re-submission Date		Date Received 2 nd Submission	
Student Name	Nguyen Binh Minh	Student ID	BH02386
Class	SE08102	Assessor name	Nguyen Thanh Trieu
Plagiarism Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.			
Student Declaration I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.			
		Student's signature	Nguyen Binh Minh

Grading grid

P1	P2	P3	P4	M1	M2	D1

ASSIGNMENT GROUP WORK

Qualification	Pearson BTEC Level 5 Higher National Diploma in Computing		
Unit number and title	Unit 20: Applied Programming and Design Principles		
Submission date		Date Received 1st submission	
Re-submission Date		Date Received 2nd submission	
Group number:	Student names & codes	Final scores	Signatures
Class		Assessor name	

Plagiarism

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student Declaration

I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.

P5	P6	P7	M3	M4	D2

OBSERVATION RECORD

Student			
Description of activity undertaken			
Assessment & grading criteria			
How the activity meets the requirements of the criteria			
Student signature:		Date:	
Assessor signature:		Date:	
Assessor name:			

☐ Summative Feedback:		☐ Resubmission Feedback:
Grade:	Assessor Signature:	Date:
Internal Verifier's Comments:		
Signature & Date:		

I. Introduction	10
II. Content	10
Task 1: INVESTIGATING THE OBJECT-ORIENTED PARADIGM AND SOLID PRINCIPLES.....	10
1. The Object-Oriented Programming (OOP) Paradigm	10
2. Class Relationships in the SIMS Scenario	15
3. The S.O.L.I.D Principles	16
Task 2: Fundamental Techniques of Clean Coding.....	21
1. Breakdown of 4 Basic Clean Coding Techniques.....	21
2. Applying the 4 Techniques to a List<List<int>> Data Structure in C#	23
Task 3: System Architecture Analysis and Testing Regime	26
1. Brief Summary of the System Requirements	26
2. Diagram Analysis and Explanation	26
3. Applying SOLID Principles to the Project.....	31
4. Testing Plan	35
Task 4: Design Patterns - Definitions and Practical Applications	37
1. What are Design Patterns?.....	37
2. Creational Patterns.....	37
3. Structural Patterns	38
4. Behavioral Patterns	39
Task 5: Performance Evaluation and Justification of SOLID Principles	40
III. Conclusion	46

Figure 1: Object-Oriented Programming (OOP) Paradigm.....	10
Figure 2: Encapsulation	11
Figure 3: Abstraction	12
Figure 4: Inheritance	13
Figure 5: Polymorphism.....	14
Figure 6: Class Diagram Relationship	15
Figure 7. S.O.L.I.D	16
Figure 8: SRP	17
Figure 9: OCP	18
Figure 10: LSP	19
Figure 11: ISP	20
Figure 12: DIP	21
Figure 13: Example Code Of Meaningful Naming	22
Figure 14: Example Code of Modularity.....	22
Figure 15: Example Code of Comments	23
Figure 16: Example Code of Consistency	23
Figure 17: Meaningful Naming Code.....	24
Figure 18:Modularity Code.....	24
Figure 19:Comments Code	25
Figure 20: Consistency Code	25
Figure 21: Use Case Diagram	27
Figure 22: Class Diagram	28
Figure 23: Package Diagram	31
Figure 24: SRP Code.....	32
Figure 25: OCP Code.....	33
Figure 26: LSP Code	33
Figure 27: ISP Code.....	34
Figure 28: DIP Code	35
Figure 29: Example Code of Creational Patterns	38
Figure 30: Example Code of Structural Patterns	39
Figure 31: Example Code of Behavioral Patterns	40
Figure 32: Bad Code SRP.....	41
Figure 33: Good Code SRP	41
Figure 34: Bad Code OCP	42
Figure 35: Good Code OCP	42
Figure 36: Bad Code LSP	43
Figure 37: Good Code LSP	43
Figure 38: Bad Code ISP.....	44
Figure 39: Good Code ISP	44
Figure 40: Bad Code DIP	45
Figure 41: Good Code DIP	45

Table 1: Testing Plan Summary	36
Table 2: Sample Test Cases.....	37
Table 3: Comprehensive SOLID	46

I. Introduction

In the contemporary educational landscape, data management efficiency, system scalability, and software maintainability are critical pillars for academic institutions. As universities expand and student enrollment grows, legacy administrative systems often struggle under the weight of large datasets and evolving functional requirements. To address these challenges, modern software engineering heavily relies on robust architectural paradigms, strictly guided by proven design principles and patterns.

This report presents a comprehensive architectural and design analysis for the development of the **Student Information Management System (SIMS)**. The primary goal of SIMS is to modernize the university's student information operations—streamlining student registration, course management, and secure role-based authentication—while guaranteeing key non-functional requirements such as scalability, performance, security, and high reliability.

The application is architected using **ASP.NET Core** and handles persistent storage through a flat, large-scale **CSV dataset**. To ensure that the system remains clean, adaptable, and easily testable, the design extensively incorporates **Object-Oriented Programming (OOP) paradigms**, **SOLID development principles**, and **Clean Coding techniques**. Furthermore, multiple **Creational, Structural, and Behavioral design patterns** are integrated to resolve complex structural dependencies and workflow controls.

II. Content

Task 1: INVESTIGATING THE OBJECT-ORIENTED PARADIGM AND SOLID PRINCIPLES

1. The Object-Oriented Programming (OOP) Paradigm

Definition of OOP:

Object-Oriented Programming (OOP) is a programming paradigm built upon the concept of "objects." In this approach, software design is organized around data (or objects) rather than functions and logic. An object is a self-contained component that encompasses both data (represented as attributes or properties) and behaviors (represented as methods or functions). OOP is highly favored in modern software engineering because it maps real-world entities into software representations, making complex systems—like the Student Information Management System (SIMS)—easier to conceptualize, develop, and maintain

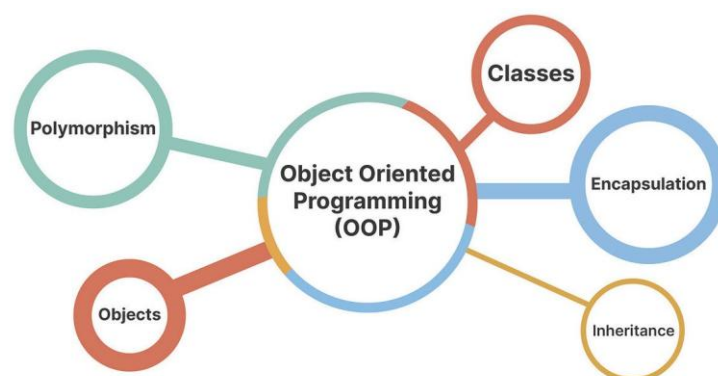


Figure 1: Object-Oriented Programming (OOP) Paradigm

The Four Fundamental Principles of OOP

To fully leverage the OOP paradigm, developers must adhere to its four core principles: Encapsulation, Inheritance, Abstraction, and Polymorphism.

a. Encapsulation

Definition:

Encapsulation is one of the fundamentals principles of OOP. It teaches us to hide the internal details of a object and expose only what is necessary through public interfaces. This means that an object's private attributes and methods remain protected, and their access is controlled by public methods like getters and setters. In this way, the internal state stays safe from unwanted changes, maintaining the integrity of the data.

Example

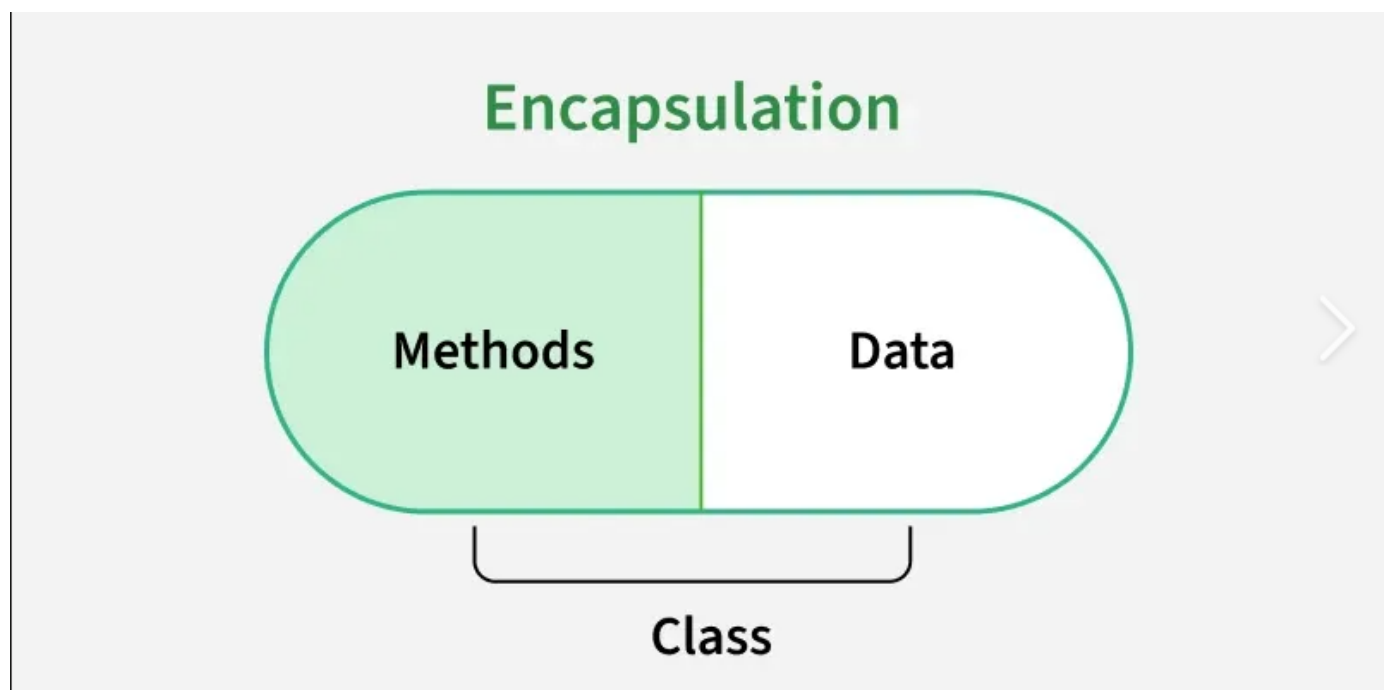


Figure 2: Encapsulation

In this example, the account balance is protected(private), and only can be modified through the controlled methods. This guarantees that the balance changes are made in a safe and correct manner.

Benefits:

Data security: prevents sensitive information from being accessed or modified directly.

Easy maintenance: changes to internal details do no affect the external code that interacts with the object.

Modularity: each object becomes an independent unit, improving the organization of the system.

b. Abstraction

Definition:

Abstraction is the process of **hiding complexity** and **exposing only the essential details** of an object. Instead of revealing all the internal implementation, only relevant operations are made available externally. This helps developers focus on the primary functionalities of a class or object, without worrying about the internal implementation details.

Example:

Consider a payment system with different payment methods like credit card, PayPal, and bank transfer. We can use an interface or an abstract class called Payment, where the specific details of each payment method are hidden. The idea is to provide a common way to process payments:

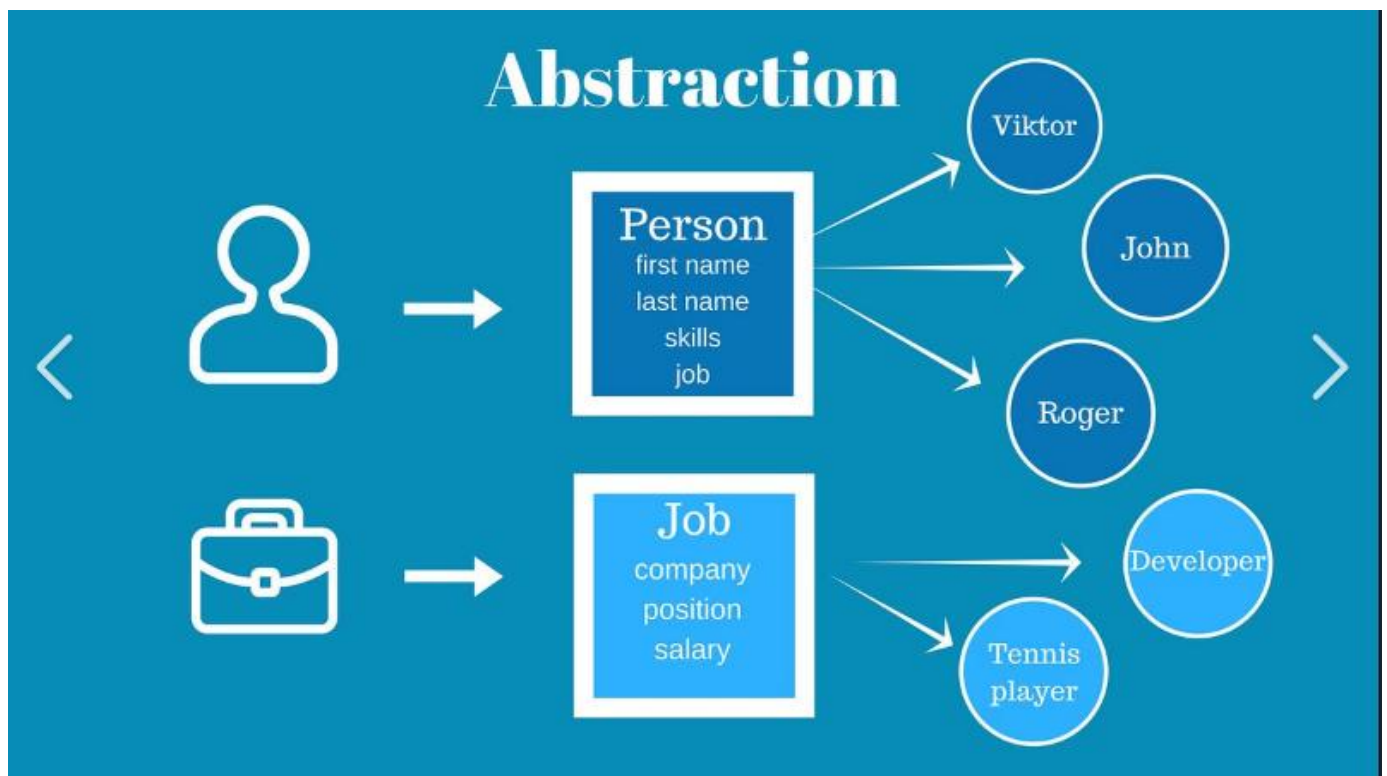


Figure 3: Abstraction

Benefits:

Simplification: focus on the essential aspects, making objects easier to use and understand.

Flexibility: different implementations can be created without changing the external interface.

Easier maintenance: changes to the implementation do not affect the code that uses the abstraction.

c. Inheritance

Definition:

Inheritance is the mechanism by which one class **inherits the characteristics (attributes and methods)** of another class. The class that inherits is called a **subclass** or **derived class**, while the class that is inherited from is called the **superclass** or **base class**. With inheritance, the subclass can reuse the code from the superclass, avoiding duplication and promoting code reuse.

Practical Example:

Let's consider a scenario with a superclass Vehicle and two subclasses Car and Motorcycle:

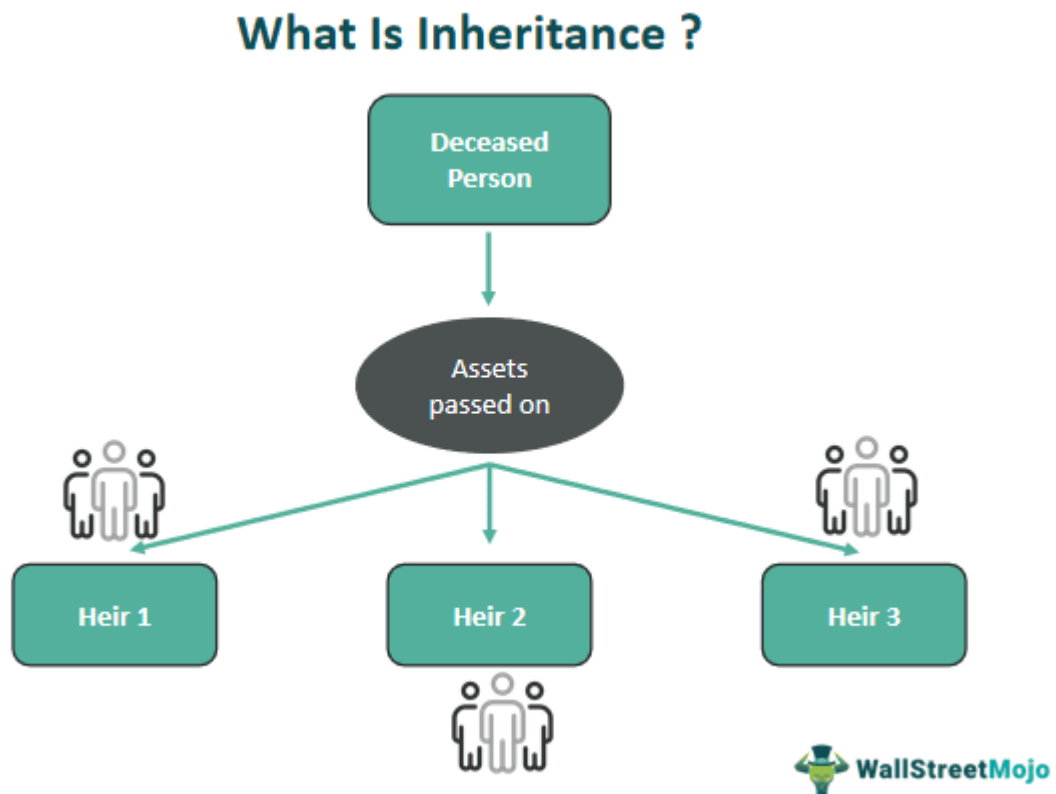


Figure 4: Inheritance

In this example, both Car and Motorcycle inherit the `start()` method from the Vehicle class. The subclasses can also have their own specific behaviors, such as `openDoor()` for Car and `raiseKickstand()` for Motorcycle.

Benefits

Code reuse: prevents duplication by allowing subclasses to reuse methods and attributes from the superclass.

Easier extension: common behaviors can be centralized in the superclass, while subclasses add specific functionality.

Hierarchical organization: reflects real-world relationships, making the code structure more logical.

d. Polymorphism

Definition:

Polymorphism allows a single interface or method to have **multiple forms** of implementation or execution. In practice, this means that different objects can respond to the same message or method call in different ways, making the code more flexible and extensible.

Polymorphism can occur in two main forms:

- **Method overloading (compile-time polymorphism):** the same method with different signatures.
- **Method overriding (runtime polymorphism):** methods with the same name but different implementations in subclasses.

Practical Example:

Going back to the payment example, we can see polymorphism in action when using the same `processPayment()` method call, but with different behaviors depending on the payment method:

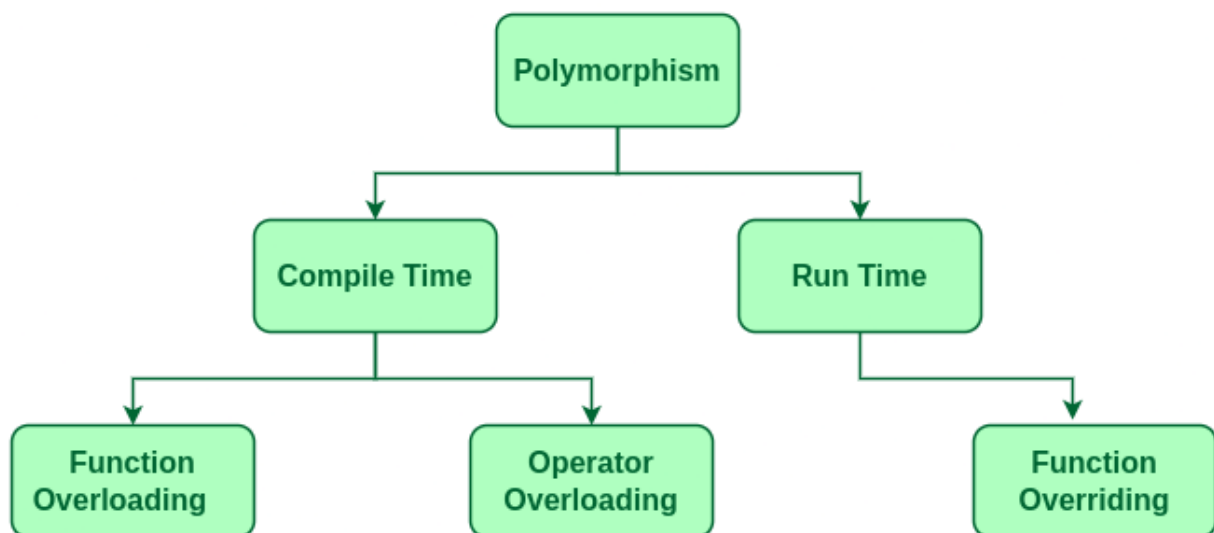


Figure 5: Polymorphism

Benefits

Flexibility: allows a method to have different behaviors depending on the object that implements it.

Extensibility: makes it easier to add new functionality without modifying existing code.

Code reuse: enables general methods to operate on different types of objects, making the system more modular.

2. Class Relationships in the SIMS Scenario

Based on the requirement to build a Student Information Management System (SIMS), here is how you identify and explain the class relationships for your Class Diagram.

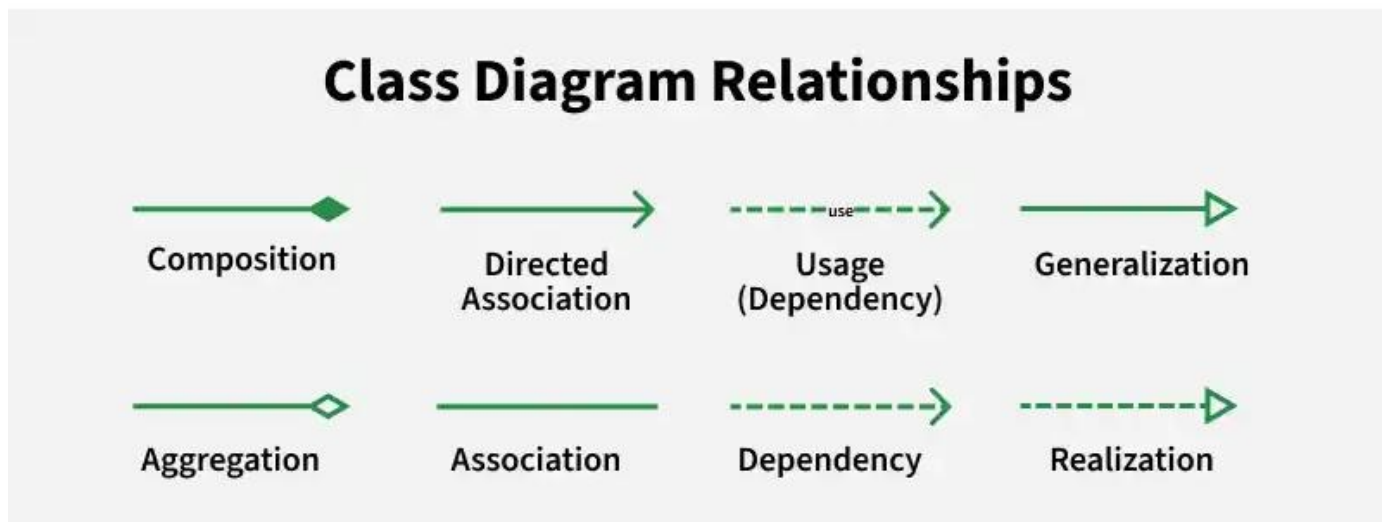


Figure 6: Class Diagram Relationship

Association

Definition: A general relationship where two classes interact or communicate with each other, but have no ownership over one another. Both have their own independent lifecycles.

SIMS Application: The relationship between a **Student** and a **Course**.

Explanation: A student is assigned to courses based on their academic program. The student uses the course, and the course contains the student. If a student drops out, the course still exists. If the course is canceled, the student still exists.

Aggregation

Definition: A specialized form of association that represents a "has-a" or "whole-part" relationship. However, the child class can exist completely independently of the parent class.

SIMS Application: The relationship between a **Department** (or University) and a **Faculty Member**.

Explanation: A department *has* faculty members who require secure user authentication. However, if the department is restructured or dissolved, the faculty members still exist as individuals and might be reassigned elsewhere.

Composition

Definition: A strict "part-of" relationship where the child cannot exist without the parent. If the parent object is destroyed, the child objects are also destroyed.

SIMS Application: The relationship between a **Student** and an **Academic Record**.

Explanation: The system must capture and store essential student information, including personal details and academic records. An academic record strictly belongs to one specific student. If a student's profile is entirely deleted from the system, their academic record must also be deleted, as it has no meaning on its own.

3. The S.O.L.I.D Principles



Figure 7. S.O.L.I.D

S - Single Responsibility Principle (SRP)

Definition: A class should have one, and only one, reason to change, meaning it should only have one job.

Application: In SIMS, instead of putting all actions into the Student class (holding personal info, saving to DB, printing transcripts), we split it into 3 separate classes:

Student: Holds personal attributes (name, age, class).

StudentRepository: Handles saving/reading data from the Database.

GradePrinter: Handles formatting and exporting transcripts.

Pros, Cons & Difficulties:

Pros: Highly readable code, easy to write independent Unit Tests, modifying one feature won't break another.

Difficulties: Hard to define the absolute boundary of a "single responsibility". As the system grows, the number of classes increases rapidly, which can feel fragmented if over-divided.

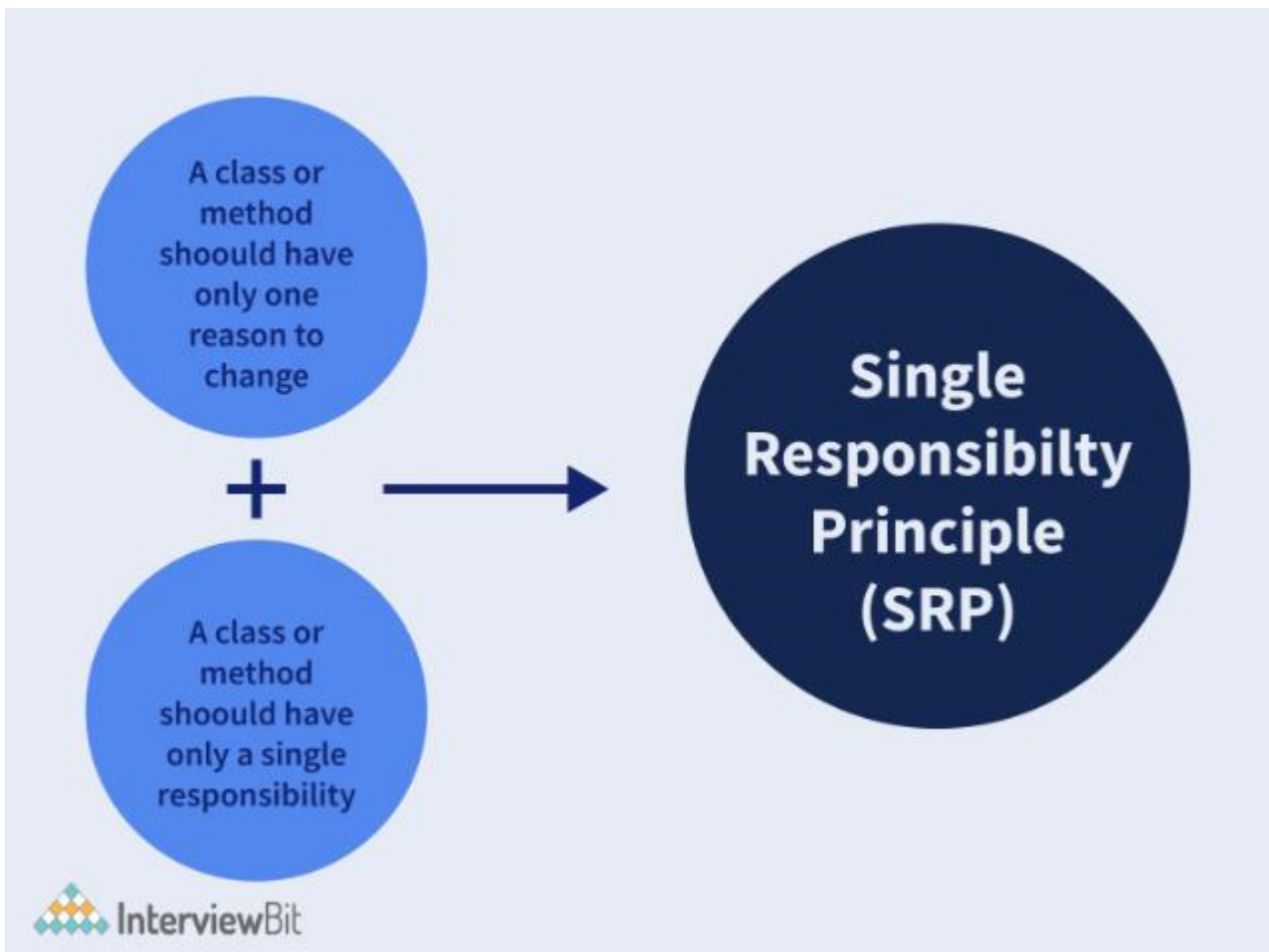


Figure 8: SRP

O - Open/Closed Principle (OCP)

Definition: Software entities (classes, modules, functions) should be open for extension but closed for modification.

Application: When calculating GPA, SIMS might have different grading rules (Regular program uses a 4.0 scale, High-Quality program uses a 10.0 scale).

Violating code: Using if-else in the GPACalculator class to check the program type. Adding a new program requires modifying existing code, risking bugs.

Solution: Create a GPACalculatorStrategy interface with a calculate() method. Each program gets its own class implementing this interface (RegularGPACalculator, HighQualityGPACalculator). To add a new program, just write a new class without touching the old code.

Pros, Cons & Difficulties:

Pros: Eliminates the risk of breaking stable features when upgrading the system.

Difficulties: Requires excellent architectural design thinking to foresee changes and create suitable interfaces, increasing initial code complexity.

The open/ closed principle (OCP) Example

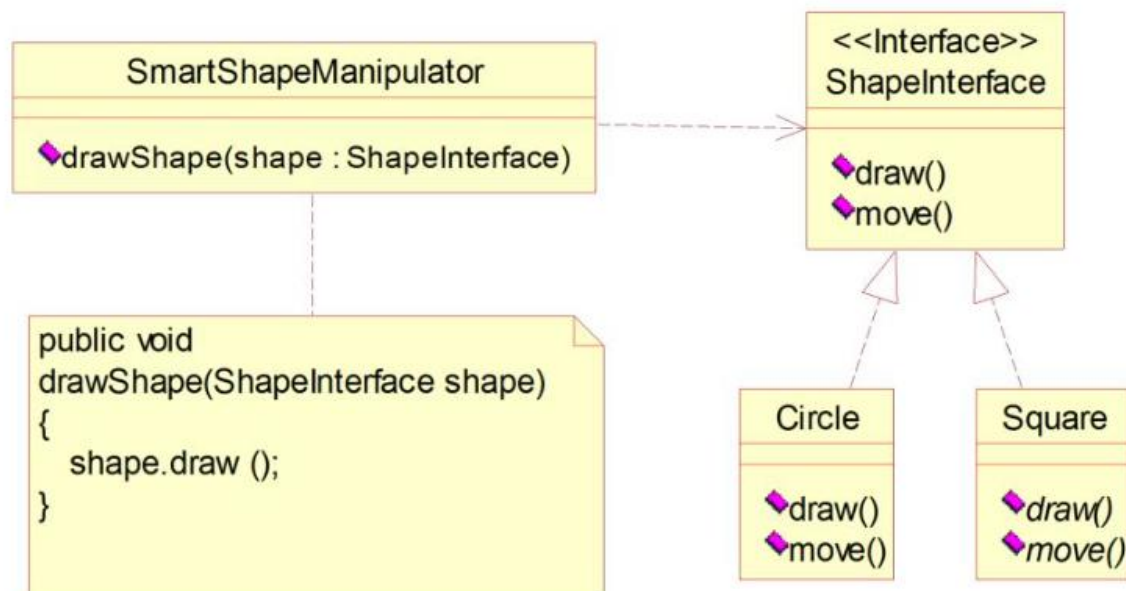


Figure 9: OCP

L - Liskov Substitution Principle (LSP)

Definition: Objects of a superclass shall be replaceable with objects of its subclasses without breaking the application.

Application: Suppose the Student superclass has a `payTuition()` method. We create an `ExchangeStudent` subclass (who gets a full scholarship and doesn't need to pay tuition).

If `ExchangeStudent` overrides `payTuition()` and throws an `Exception("No tuition needed")`, the program iterating through students to collect money will crash. This violates LSP.

Solution: Extract the tuition payment into a separate `TuitionPayable` interface. Only students who actually need to pay will implement this interface, ensuring subclasses never alter the expected behavior of the superclass.

Pros, Cons & Difficulties:

Pros: Ensures proper use of inheritance, eliminating hidden logic bugs at runtime.

Difficulties: Easy to violate when trying to force real-world objects that seem similar but behave fundamentally differently into the same inheritance tree.

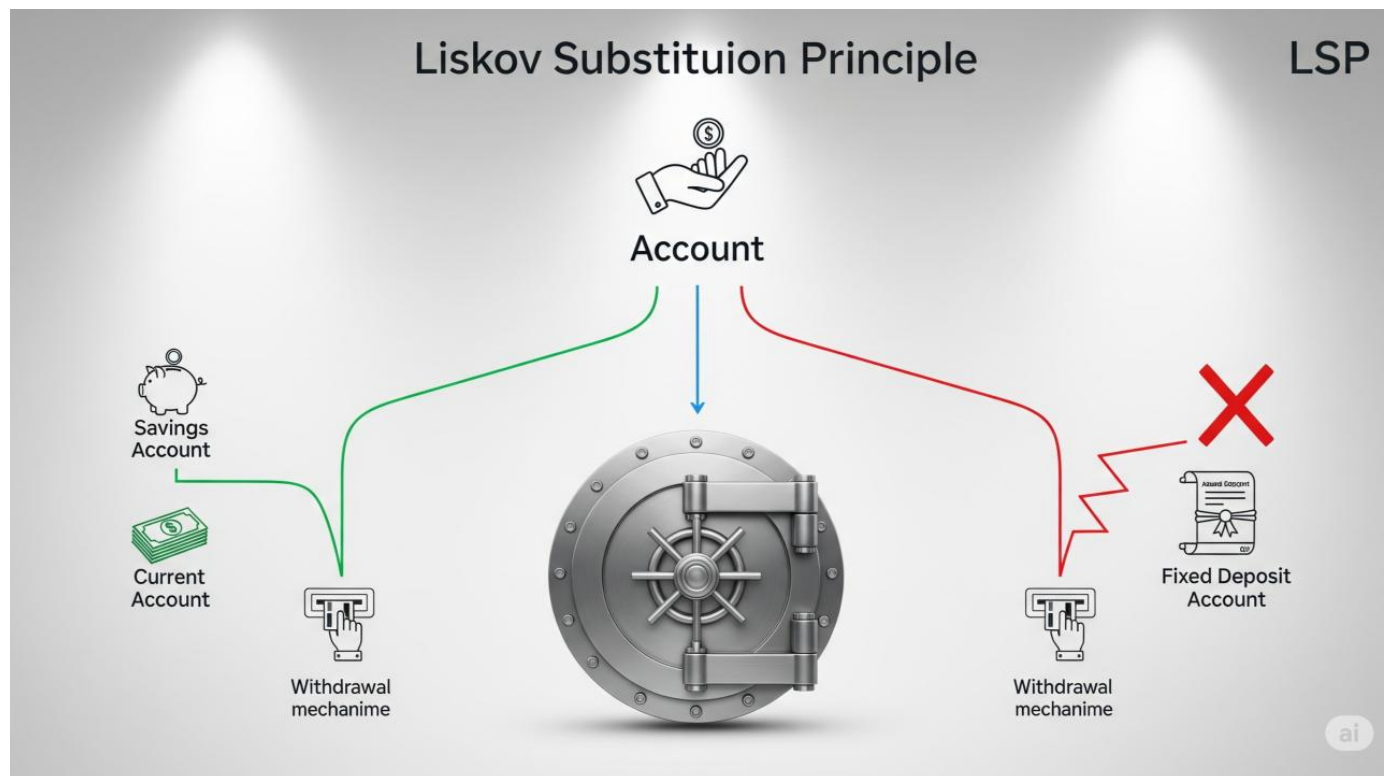


Figure 10: LSP

I - Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend upon interfaces that they do not use. It's better to have many smaller, specific interfaces than one large, general interface.

Application: In SIMS, if we create a massive StudentOperations interface with: study(), takeExam(), submitThesis(), and manageFaculty().

A normal student would have to implement an empty manageFaculty() method, while an admin would implement empty study() methods.

Solution: Break it down into specific interfaces: Learnable (has study, takeExam), Graduatable (has submitThesis), and Managable (has manageFaculty). Classes only implement what they actually need.

Pros, Cons & Difficulties:

Pros: Keeps interfaces clean and focused; avoids empty/dummy methods just to satisfy a contract.

Difficulties: Increases the number of interfaces, requiring highly organized directory structures and naming conventions to avoid clutter.

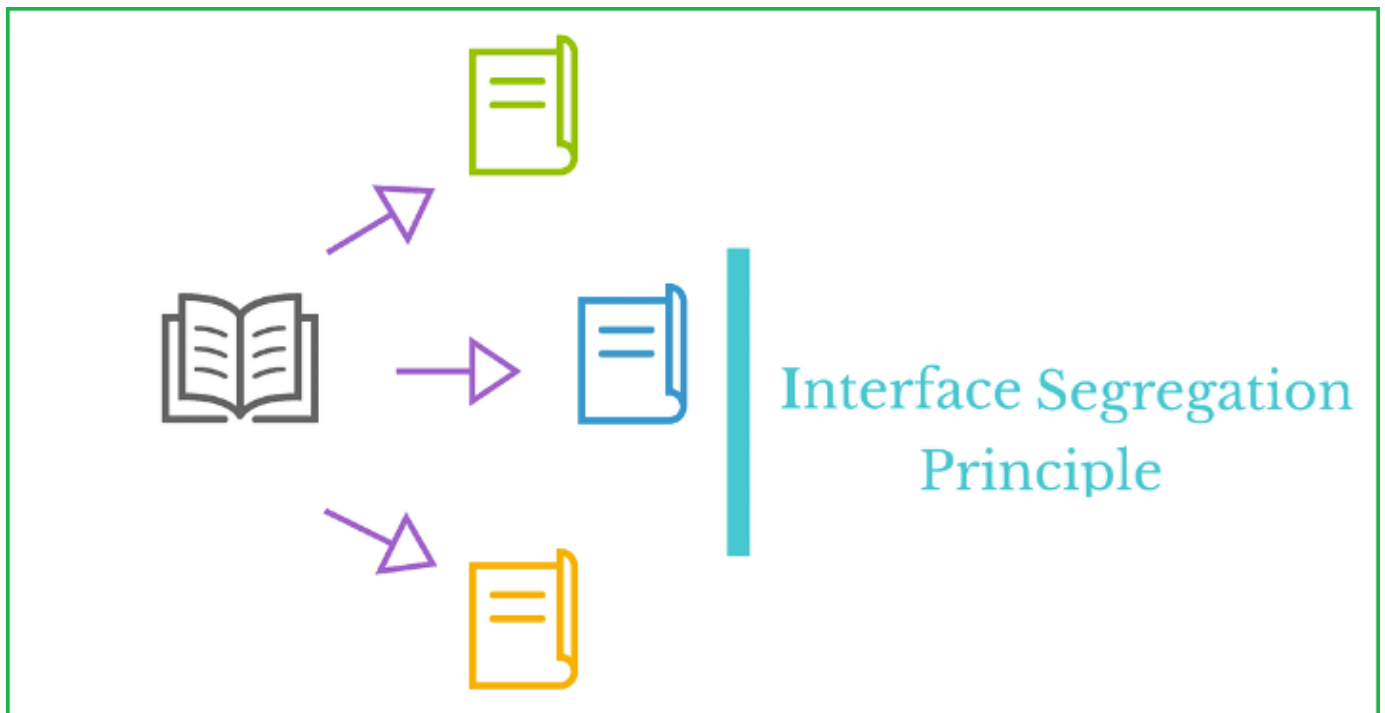


Figure 11: ISP

D - Dependency Inversion Principle (DIP)

Definition:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

Application: The high-level StudentService needs to save student data.

Violating code: Instantiating MySQLDatabase db = new MySQLDatabase() directly inside StudentService. This hard-codes the dependency. If we switch to MongoDB, we must rewrite StudentService.

Solution: Create an abstract DatabaseInterface. StudentService calls methods via this interface. Both MySQLDatabase and MongoDBDatabase implement it. Now, we can swap databases without modifying a single line in StudentService.

Pros, Cons & Difficulties:

Pros: Extremely high flexibility and reusability. Makes creating mock objects for Unit Testing straightforward.

Difficulties: Hard to grasp for beginners. Often requires learning and integrating Dependency Injection (DI) frameworks to manage object lifecycles automatically.

Dependency inversion principle

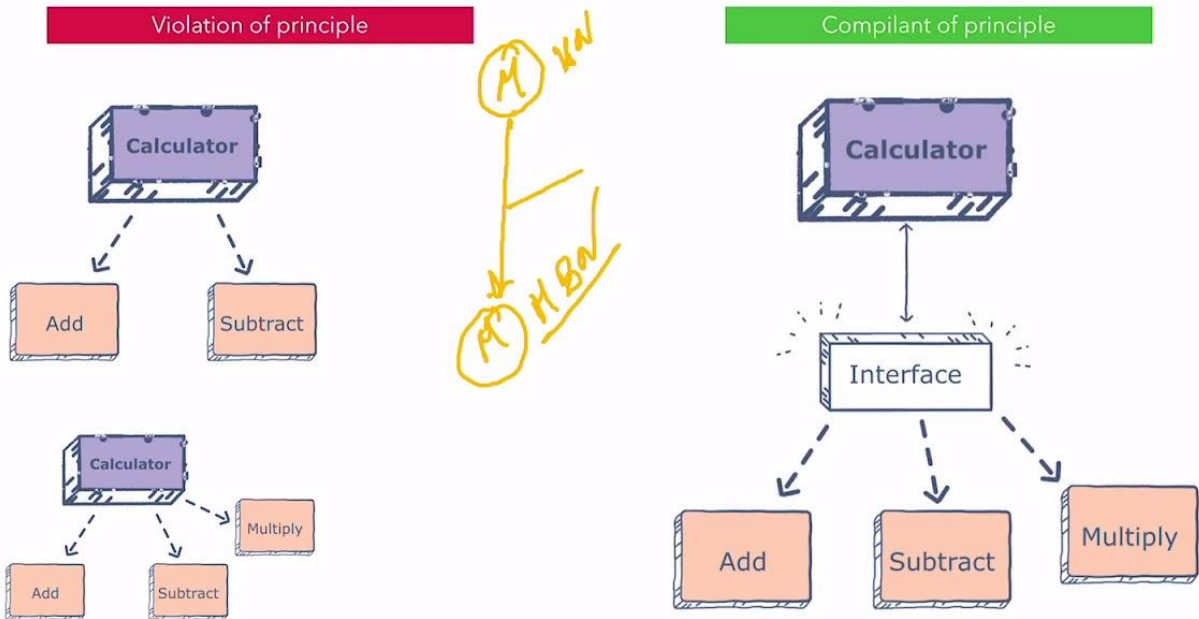


Figure 12: DIP

Task 2: Fundamental Techniques of Clean Coding

1. Breakdown of 4 Basic Clean Coding Techniques

1.1. Meaningful Naming

What it is: The practice of assigning descriptive names to variables, functions, classes, or parameters that accurately reflect their purpose and content. Your code should read like natural text.

How it helps: It makes the code "self-documenting." When you or a team member read the code months later, everyone will immediately understand what a variable is used for without having to trace through the entire logic flow. This saves a massive amount of time during debugging and maintenance.

Example:

```

// Bad (Meaningless)
int d;
bool is0;

// Good (Meaningful Naming)
int elapsedDays;
bool isOrderCompleted;

```

Figure 13: Example Code Of Meaningful Naming

1.2. Modularity

What it is: The technique of breaking down a large, complex system into smaller, independent, and manageable parts (functions, classes, or modules). Each function or class should have a **single responsibility** (Single Responsibility Principle).

How it helps: It makes code highly reusable. If an error occurs, you can easily isolate and fix it in a small module without worrying about breaking the entire system. Writing Unit Tests also becomes much easier.

Example:

```

1 // Bad: One function doing too many things (fetching, calculating, printing)
2 // public void ProcessData() { /* 100 lines of mixed logic */ }
3
4 // Good: Broken down into specific, single-purpose functions
5 // public List<Data> FetchData() { ... }
6 // public double CalculateAverage(List<Data> data) { ... }
7 // public void PrintReport(double average) { ... }

```

Figure 14: Example Code of Modularity

1.3. Comments

What it is: In Clean Code philosophy, good code is its own documentation. Therefore, the best comments do not explain *what* the code is doing, but rather **why the code is written that way**, or to provide specific business logic context.

How it helps: It provides readers with context about obscure technical decisions, workarounds for third-party libraries, or warnings about the negative impacts of modifying that specific block of code.

Example:

```

public class Calculator
{
    // Bad: Magic number 0.05 (what is this? tax? discount?)
    public double CalculateTotal(double price)
    {
        return price + (price * 0.05);
    }

    // Good: Naming the constant makes the code self-documenting
    private const double SalesTaxRate = 0.05;

    public double CalculateTotalOptimized(double price)
    {
        return price + (price * SalesTaxRate);
    }
}

```

Figure 15: Example Code of Comments

1.4. Consistency

What it is: Adhering to a single coding standard throughout the entire project. This includes indentation, bracket placement {}, and naming conventions (e.g., in C#: PascalCase for Classes/Methods, camelCase for local variables).

How it helps: It reduces "cognitive load." When your brain gets used to a uniform format, you can read code much faster because you aren't distracted by trivial formatting differences.

Example:

```

1  // Consistent curly brace placement following C# standards
2  public void DisplayInfo()
3  {
4      if (isValid)
5      {
6          // Do something
7      }
8  }

```

Figure 16: Example Code of Consistency

2. Applying the 4 Techniques to a List<List<int>> Data Structure in C#

Problem and Data Structure

The Problem: Write a program to manage the scores of student groups.

Data Structure: The data structure is a list containing smaller lists (List<List<int>> – acting similarly to a 2D array or a List<ArrayList>, but strongly typed and optimized in C#).

Requirement: Process and calculate the average score for each group.

Technique: Meaningful Naming

```
1 // The method name and variable clearly describe their functions
2 public void ProcessAllGroups(List<List<int>> studentGroupsScores)
3 {
4     if (studentGroupsScores == null) return;
5     // ...
6 }
```

Figure 17: Meaningful Naming Code

Detailed Explanation:

Where it is applied: In the method signature and the input parameter studentGroupsScores.

How it is applied: Instead of using meaningless names like List<List<int>> a, we use studentGroupsScores to accurately describe that these are group scores. This makes the code self-documenting and tells the reader exactly what data the variable holds.

Technique: Modularity

```
7 // Method 1: Iterating through the list
8 public void ProcessAllGroups(List<List<int>> studentGroupsScores)
9 {
10     // ...
11     var currentGroup = studentGroupsScores[i];
12
13     // Calling a specialized function for calculation
14     double average = CalculateGroupAverage(currentGroup);
15     Console.WriteLine($"Group {i + 1} average score: {average:F2}");
16 }
17
18 // Method 2: Single Responsibility of calculating the average
19 private double CalculateGroupAverage(List<int> scores)
20 {
21     double totalScore = 0;
22     foreach (var score in scores)
23     {
24         totalScore += score;
25     }
26     return totalScore / scores.Count;
27 }
```

Figure 18:Modularity Code

Detailed Explanation:

Where it is applied: Separating the logic into two distinct functions (ProcessAllGroups and CalculateGroupAverage).

How it is applied: The process of iterating through the list and the process of calculating the average are separated. This modular approach follows the Single Responsibility Principle, making it much easier to isolate logic, maintain the codebase, and write independent Unit Tests.

Technique: Comments

```
28 // We must skip groups with no students (Count == 0)
29 // to prevent a DivideByZeroException when calculating the average.
30 if (currentGroup.Count > 0)
31 {
32     double average = CalculateGroupAverage(currentGroup);
33 }
```

Figure 19: Comments Code

Detailed Explanation:

Where it is applied: Right above the conditional statement `if (currentGroup.Count > 0)`.

How it is applied: A comment is added to explain *why* we must check `Count > 0` (to avoid a divide-by-zero error). Clean code uses comments to explain the business context or the "why" behind a decision, rather than stating the obvious "what" the code is doing.

Technique: Consistency

```
34 public void ProcessAllGroups(List<List<int>> studentGroupsScores)
35 {
36     for (int i = 0; i < studentGroupsScores.Count; i++)
37     {
38         // Consistent use of 'var'
39         var currentGroup = studentGroupsScores[i];
40
41         if (currentGroup.Count > 0)
42         {
43             double average = CalculateGroupAverage(currentGroup);
44         }
45     }
46 }
```

Figure 20: Consistency Code

Detailed Explanation:

Where it is applied: Throughout the structural formatting, naming conventions, and variable declarations.

How it is applied: The code demonstrates consistency by adhering to standard C# conventions. This includes using PascalCase for methods, consistent use of the `var` keyword for local variables, and proper indentation. This uniformity significantly reduces cognitive load for anyone reading the code.

Task 3: System Architecture Analysis and Testing Regime

1. Brief Summary of the System Requirements

The problem requires building a secure **Student Information Management System (SIMS)**. This system needs to serve three main user groups (Administrator, Student, and Faculty) with different privileges. The key characteristics of the system include:

Using CSV files for data storage (instead of a traditional SQL database).

Being designed with a multi-layered architecture (Web UI, Business Logic, Data Access).

Strictly adhering to **SOLID** principles to ensure maintainability, scalability, and testability.

Implementing a Role-Based Access Control (RBAC) mechanism.

2. Diagram Analysis and Explanation

A. Use Case Diagram

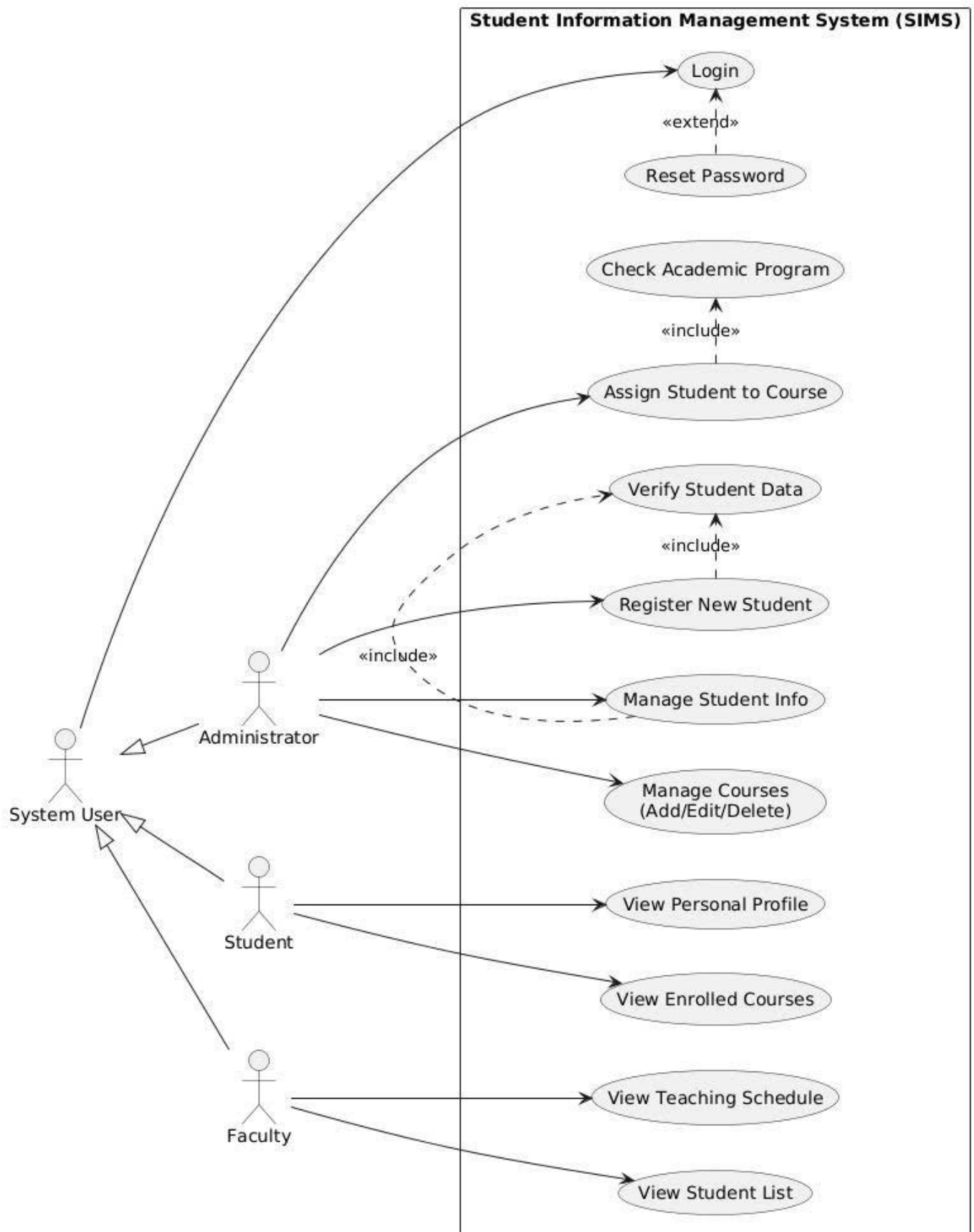


Figure 21: Use Case Diagram

Purpose: This diagram answers the question *"Who can do what in the system?"* It describes the functionalities from the perspective of the users.

Components & Relationships:

Actors: There is a general System User actor. The three specific actors (Administrator, Student, and Faculty) inherit from it via **generalization**. This means all common features (like Login) are available to all three.

Include: Represented by a dashed arrow with <<include>>. For example, the Register New Student use case *must* call Verify Student Data. This means data verification is a mandatory step for registration.

Extend: Represented by a dashed arrow with <<extend>>. For example, Reset Password points to Login. This means when on the Login screen, the user *can optionally* choose the reset password feature.

Workflow:

Admin: Manages student information, adds/edits/deletes courses, and assigns students to classes.

Student: Views their personal profile and enrolled courses.

Faculty: Views their teaching schedule and the student list for their classes.

B. Class Diagram

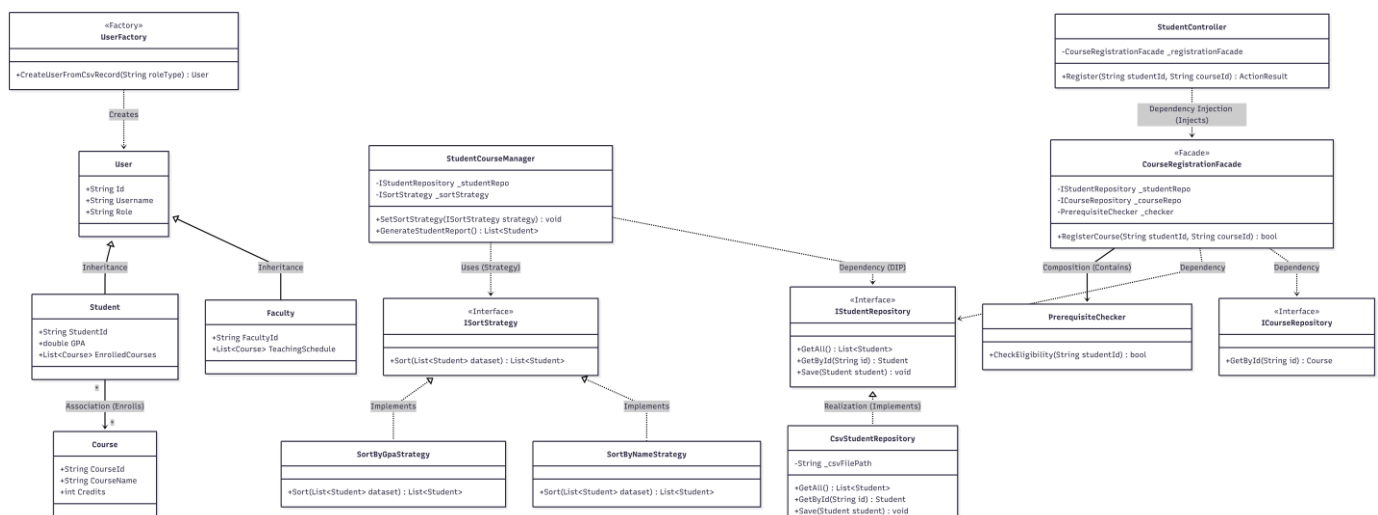


Figure 22: Class Diagram

Purpose: This updated Class Diagram illustrates the static architectural structure of the Student Information Management System (SIMS). It highlights how the system achieves loose coupling through the use of interfaces and demonstrates the practical integration of multiple Design Patterns (Factory, Strategy, and Facade) to ensure high scalability, maintainability, and adherence to SOLID principles.

Core Components & Relationships:

Entities and Inheritance: At the domain level, User acts as the base class containing common attributes (Id, Username, Role). Both Student and Faculty inherit from User, promoting code reusability. Additionally, there is an explicit **Association** relationship between Student and Course, indicating that a student can enroll in multiple courses.

Creational Pattern - Factory Method: The UserFactory handles the complex logic of object creation (e.g., CreateUserFromCsvRecord). Instead of scattering the new keyword throughout the application, the factory centralizes instantiation, making the code cleaner and easier to test.

Behavioral Pattern - Strategy: The StudentCourseManager utilizes the ISortStrategy interface rather than hard-coding sorting algorithms. By implementing concrete strategies like SortByGpaStrategy and SortByNameStrategy, the system can dynamically change how student data is sorted at runtime. This perfectly satisfies the Open/Closed Principle (OCP)—we can add new sorting methods in the future without modifying the StudentCourseManager.

Structural Pattern - Facade: The CourseRegistrationFacade is introduced to simplify the course registration process for the Web UI layer. The StudentController only interacts with this single Facade class via Dependency Injection. Internally, the Facade orchestrates the complex workflow: it uses Composition to call the PrerequisiteChecker (to verify eligibility) and coordinates with both the IStudentRepository and ICourseRepository to finalize the registration.

Dependency Inversion Principle (DIP): High-level business modules (StudentCourseManager and CourseRegistrationFacade) do not depend on low-level data access classes like CsvStudentRepository. Instead, they depend entirely on abstractions (IStudentRepository, ICourseRepository). The CsvStudentRepository realizes (implements) the interface, meaning the underlying storage mechanism can be upgraded from CSV to SQL in the future without breaking any core business logic.

C. Package Diagram

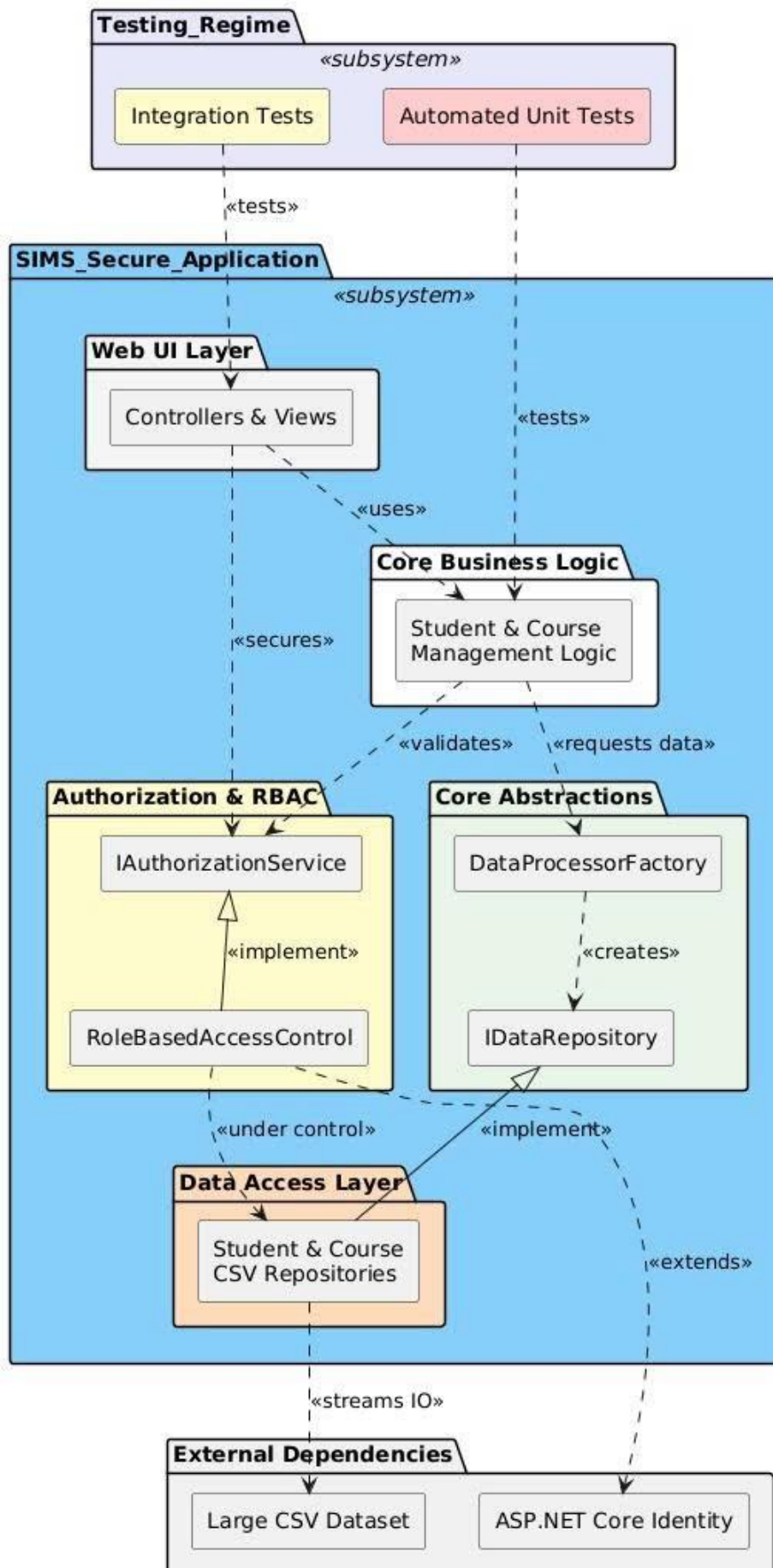


Figure 23: Package Diagram

Purpose: Helps modularize the system, managing the macro-architecture of the project and demonstrating how large modules interact with each other.

Components:

SIMS_Secure_Application: The main package containing the entire application source code, subdivided into smaller sub-packages (Web UI, Core Business, Data Access, Authorization, Core Abstractions).

Testing_Regime: Contains Automated Unit Tests and Integration Tests.

External Dependencies: Contains external resources/libraries (Large CSV Dataset, ASP.NET Core Identity).

Relationships: Dashed arrows indicate dependencies. Web UI depends on Core Business. Core Business depends on Core Abstractions. Testing depends on the entire application package.

3. Applying SOLID Principles to the Project

S - Single Responsibility Principle (SRP):

Application: Each class has only one specific job. The UI (StudentController) only handles web routing. StudentCourseManager only handles business logic. CsvStudentRepository only reads/writes CSV files, completely unaware of enrollment logic.

Where it is applied in code: The separation of the Controller, Manager, and Repository layers.

Code Illustration:


```

1  // 1. UI Layer: Only handles HTTP requests and routing
   0 references | 0 references
2  public class StudentController : ControllerBase
3  {
4      0 references | 0 references
       private readonly IStudentRegistration _studentService;
       0 references | 0 references
5      public IActionResult Register(Student student) { ... }
6  }
7
8  // 2. Business Logic Layer: Only handles validation and business rules
   0 references | 0 references
9  public class StudentCourseManager : IStudentRegistration
10 {
11     0 references | 0 references
       public void VerifyStudentData(Student student) { /* Validation logic */ }
12 }
13
14 // 3. Data Access Layer: Only handles File I/O operations
   0 references | 0 references
15 public class CsvStudentRepository : IStudentRepository
16 {
17     0 references | 0 references
       public void Save(Student student) { /* CSV writing logic */ }
18 }

```

Figure 24: SRP Code

Benefit: Modifying one component (e.g., updating the UI framework) does not break or affect the logic in other layers.

O - Open/Closed Principle (OCP):

Application: The system heavily utilizes Interfaces (IStudentRepository, ICourseRepository).

Where it is applied in code: The core abstractions layer defining contracts for data storage.

Code Illustration:


```

19 // Base Interface (Closed for modification)
   3 references | 3 references
20 public interface IStudentRepository
21 {
22     2 references | 2 references
   void Save(Student student);
23 }
24
25 // Existing Feature (Working perfectly)
   1 reference | 1 reference
26 public class CsvStudentRepository : IStudentRepository
27 {
28     1 reference | 1 reference
   public void Save(Student student) { /* Save to data.csv */ }
29 }
30
31 // EXTENSION (Open for extension): Adding SQL support later without touching CsvStudentRepository or the Manager
   0 references | 0 references
32 public class SqlStudentRepository : IStudentRepository
33 {
34     1 reference | 1 reference
   public void Save(Student student) { /* INSERT INTO Students... */ }
35 }

```

Figure 25: OCP Code

Benefit: Open for extension, closed for modification. If you later want to scale to a SQL Server database, you simply create a new class implementing the existing interface, without needing to rewrite a single line of code in StudentCourseManager.

L - Liskov Substitution Principle (LSP):

Application: Subclasses or implementing classes must be substitutable for their base types. Here, StudentCourseManager relies on IStudentRepository and works correctly regardless of whether the actual injected object is a CSV repository or a SQL repository.

Where it is applied in code: Passing different repository implementations into the Manager without crashing the application.

Code Illustration:

```

36 // The Manager expects the Interface
37 IStudentRepository activeRepository;
38
39 // Both of these can perfectly substitute the interface without causing runtime errors or unexpected behaviors
40 activeRepository = new CsvStudentRepository();
41 activeRepository.Save(newStudent); // Works fine
42
43 activeRepository = new SqlStudentRepository();
44 activeRepository.Save(newStudent); // Works perfectly, same expected outcome

```

Figure 26: LSP Code

Benefit: Ensures system consistency and reliability when using polymorphism.

I - Interface Segregation Principle (ISP):

Application: Instead of forcing everything into a single, bloated IManager interface, the design segregates functionalities into ICourseManagement and IStudentRegistration.

Where it is applied in code: The definitions of specific, role-based interfaces in the Core Abstractions layer.

Code Illustration:

```
46 // BAD: A bloated interface forcing unused methods
47 // public interface IUniversityManager { void RegisterStudent(); void DeleteCourse(); }
48
49 // GOOD: Segregated, focused interfaces
50 // 2 references | 2 references
51 public interface IStudentRegistration
52 {
53     // 0 references | 0 references
54     void RegisterNewStudent(Student student);
55     // 1 reference | 1 reference
56     void VerifyStudentData(Student student);
57 }
58
59 // 0 references | 0 references
60 public interface ICourseManagement
61 {
62     // 0 references | 0 references
63     void AddCourse(Course course);
64     // 0 references | 0 references
65     void AssignStudentToCourse(string studentId, string courseId);
66 }
```

Figure 27: ISP Code

Benefit: Classes/Controllers only depend on the methods they actually use. StudentController only injects IStudentRegistration and is not forced to implement or depend on course management features.

D - Dependency Inversion Principle (DIP):

Application: High-level modules (StudentCourseManager) do not depend on low-level modules (CsvStudentRepository). Both depend on abstractions (IStudentRepository).

Where it is applied in code: The constructor of StudentCourseManager using Dependency Injection.

Code Illustration:

```

61 public class StudentCourseManager : IStudentRegistration, ICourseManagement
62 {
63     // High-level module depends on ABSTRACTIONS, not concrete classes
64     // 2 references | 2 references
65     private readonly IStudentRepository _studentRepo;
66     // 1 reference | 1 reference
67     private readonly ICourseRepository _courseRepo;
68
69     // Dependencies are injected via constructor
70     // 0 references | 0 references
71     public StudentCourseManager(IStudentRepository studentRepo, ICourseRepository courseRepo)
72     {
73         _studentRepo = studentRepo;
74         _courseRepo = courseRepo;
75     }
76
77     // 1 reference | 1 reference
78     public void RegisterNewStudent(Student student)
79     {
80         // Calls the abstraction, totally unaware if it's a CSV or SQL underneath
81         _studentRepo.Save(student);
82     }
83 }

```

Figure 28: DIP Code

Benefit: Dramatically reduces tight coupling and lays the foundation for seamlessly applying Dependency Injection (DI) frameworks in ASP.NET Core.

4. Testing Plan

Based on the Testing_Regime package in the Package Diagram, the system's architecture makes it highly testable. The testing plan is structured into three main phases:

Testing Plan Summary

Test Phase	Objective	Scope	Method & Tools
A. Automated Unit Tests (Top Priority)	Ensure that independent business logic and validations run correctly.	Focuses primarily on the Core Business Logic and Authorization layers.	Use Mocking libraries (like Moq in C#) to create "fake" instances of IStudentRepository. This allows testing StudentCourseManager without interacting with real CSV files.
B. Integration Tests	Verify that different components and layers work together smoothly.	The interaction between the Data Access Layer and the CSV files, or the data flow from Web UI to Business Logic.	Create dummy CSV files with a small test dataset. Verify if CsvStudentRepository correctly parses text rows into Student

			objects and saves updates properly.
C. System / End-to-End Tests	Test the overall system behavior from an end-user's perspective.	The entire application flow: URL routing, UI actions, business logic, to final data persistence.	Log in as an Administrator, add a new student via the UI, and manually open the physical CSV file to verify the appended record.

Table 1: Testing Plan Summary

Sample Test Cases

To ensure the reliability of the SIMS application, the following test cases will be executed across the different testing phases:

Test Case ID	Test Type	Scenario / Description	Pre-conditions	Test Steps	Expected Result
TC-01	Unit Test	Prevent Duplicate Student ID Registration	A mocked IStudentRepository is injected into StudentCourseManager.	1. Attempt to register a new Student with an ID that already exists in the mock repository.	The system throws a DuplicateRecordException and halts registration.
TC-02	Unit Test	Enforce Course Capacity Limits	A mocked ICourseRepository returns a course with 0 available seats.	1. Call AssignStudentToCourse() for the full course.	The system returns false or throws a CapacityExceededException.
TC-03	Integration	CSV Data Parsing Accuracy	A dummy test_students.csv file exists with 3 valid records.	1. Initialize CsvStudentRepository with the dummy file path. 2. Call GetAll().	The method returns a List<Student> containing exactly 3 correctly populated objects.
TC-04	Integration	Persist New Data to CSV	A dummy test_students.csv file exists.	1. Call Save(newStudent). 2. Read the file stream directly.	The new student's data is appended to the bottom of the CSV file with the correct comma-separated format.
TC-05	E2E / System	Unauthorized Access to Admin Panel	User is logged in with the "Student" role.	1. Navigate to the /Admin/ManageStudents URL via the web browser.	The UI redirects the user to an "Access Denied" (403) page; no data is exposed.
TC-06	E2E / System	Successful End-to-End	User is logged in as "Administrator".	1. Fill out the "Add Student" form on the UI.	The UI shows a success message, and the new student data is visually

		Registration		2. Submit the form. 3. Open the actual data.csv file.	verified inside the data.csv file.
--	--	--------------	--	--	------------------------------------

Table 2: Sample Test Cases

Task 4: Design Patterns - Definitions and Practical Applications

1. What are Design Patterns?

Design patterns are reusable solutions to common software design problems that help developers build cleaner and more maintainable systems. They act as templates for solving issues related to object creation, structure, and behavior. Instead of complete code, they provide blueprints that can be adapted to different scenarios.

Provide standard terminology and best practices, making communication easier among developers.

Help create structured, scalable, and maintainable code by promoting proven design approaches.

Reduce development time by offering ready-made solutions to recurring problems.

2. Creational Patterns

Definition: A group of design patterns that deal with object creation mechanisms.

Purpose & Significance: Instead of instantiating objects directly using the new keyword (which causes tight coupling), this group provides more flexible object creation mechanisms. It helps hide complex initialization logic and allows the system to easily change the type of objects being created without affecting the code that uses them.

Application in SIMS: Factory Method (or Simple Factory) Based on the Class Diagram from Task 3, you already have a DataProcessorFactory class. Instead of the Controller having to explicitly call new CsvStudentRepository(), we delegate this responsibility to the Factory. If the system later switches to a SQL Database instead of CSV, we only need to modify the Factory.

Code Illustration:


```

1  // Standardized Interface (Core Abstraction)
   2 references | 2 references
2  ✓ public interface IStudentRepository
3  {
4      1 reference | 1 reference
       void Save(Student student);
5  }
6
7  // Concrete class that implements CSV reading/writing
   1 reference | 1 reference
8  ✓ public class CsvStudentRepository : IStudentRepository
9  {
10     1 reference | 1 reference
        public void Save(Student student)
11     {
12         Console.WriteLine($"Saving student {student.StudentId} to CSV file...");
13     }
14 }
15
16 // Creational Pattern Class: Factory
   0 references | 0 references
17 ✓ public class DataProcessorFactory
18 {
19     // Provides an initialization method, hiding the implementation details
       0 references | 0 references
20     ✓ public IStudentRepository CreateStudentRepository()
21     {
22         // Complex logic can be added here (e.g., reading configs to decide between CSV or SQL)
23         return new CsvStudentRepository(); → Tab to Jump
24     }
25 }

```

Figure 29: Example Code of Creational Patterns

3. Structural Patterns

Definition: A group of design patterns that focus on establishing relationships between objects or classes to form larger, more complex structures.

Purpose & Significance: They help incompatible software or libraries work together, or simplify a complex system by providing a single, unified interface for communication.

Application in SIMS: Facade Pattern In your system, StudentCourseManager acts as a **Facade**. The Web UI layer (Controller) only needs to call a single method here, rather than manually handling the logic to call IStudentRepository, check permissions via IAuthorizationService, and then call ICourseRepository. The Facade encapsulates and groups these complex operations together.

Code Illustration:

```

1 // Facade Class hiding the underlying system complexity
  1 reference | 1 reference
2 public class StudentCourseManager : IStudentRegistration, ICourseManagement
3 {
4     2 references | 2 references
    private readonly IStudentRepository _studentRepo;
5     2 references | 2 references
    private readonly IAuthorizationService _authService;
6
7     // Inject dependencies via the constructor
8     0 references | 0 references
    public StudentCourseManager(IStudentRepository studentRepo, IAuthorizationService authService)
9     {
10         _studentRepo = studentRepo;
11         _authService = authService;
12     }
13
14     // Web UI only needs to call this method, without knowing the internal complexity
15     0 references | 0 references
    public void RegisterNewStudent(User currentUser, Student newStudent)
16     {
17         // 1. Facade independently calls Authorization to check permissions (hiding logic from UI)
18         if (!_authService.Authorize(currentUser, "Admin"))
19         {
20             throw new UnauthorizedAccessException("Only Admins can register students.");
21         }
22
23         // 2. Facade independently calls the Repository to save data
24         _studentRepo.Save(newStudent);
25         Console.WriteLine("Registration successful via Facade!");
26     }
27 }

```

Figure 30: Example Code of Structural Patterns

4. Behavioral Patterns

Definition: A group of design patterns that manage algorithms, communication, and the assignment of responsibilities between objects.

Purpose & Significance: They help objects interact with each other loosely (loose coupling) while still ensuring the workflow runs smoothly, making it easy to change an object's behavior at runtime.

Application in SIMS: Strategy Pattern Suppose the requirements for SIMS change: Assigning a student to a course (AssignStudentToCourse) now requires different validation rules. The IT department needs to check Math scores, while the Language department checks English proficiency levels. Instead of writing a massive if...else block in StudentCourseManager, we use the **Strategy** pattern to extract these validation algorithms into separate, dedicated classes.

Code Illustration:

```

1 3 references | 3 references
2 public interface IEnrollmentValidationStrategy
3 {
4     3 references | 3 references
5     bool IsValid(Student student, Course course);
6 }
7 // Step 2: Create Concrete Strategies implementing specific algorithms
8 0 references | 0 references
9 public class ItDepartmentValidation : IEnrollmentValidationStrategy
10 {
11     2 references | 2 references
12     public bool IsValid(Student student, Course course)
13     {
14         Console.WriteLine("Checking IT prerequisites (e.g., Math score)...");
15         return true; // Assuming conditions are met
16     }
17 }
18 0 references | 0 references
19 public class LanguageDepartmentValidation : IEnrollmentValidationStrategy
20 {
21     2 references | 2 references
22     public bool IsValid(Student student, Course course)
23     {
24         Console.WriteLine("Checking Language prerequisites (e.g., English level)...");
25         return true;
26     }
27 }
28 // Step 3: Apply the Strategy in the Manager (Context)
29 0 references | 0 references
30 public class StudentCourseManager
31 {
32     // The validation algorithm can be dynamically changed at runtime
33     0 references | 0 references
34     public void AssignStudentToCourse(Student student, Course course, IEnrollmentValidationStrategy validationStrategy)
35     {
36         if (validationStrategy.IsValid(student, course))
37         {
38             Console.WriteLine($"Successfully enrolled {student.StudentId} to {course.CourseName}");
39             // Proceed to write to CSV...
40         }
41         else
42         {
43             Console.WriteLine("Student does not meet the prerequisites.");
44         }
45     }
46 }

```

Figure 31: Example Code of Behavioral Patterns

Task 5: Performance Evaluation and Justification of SOLID Principles

S - Single Responsibility Principle (SRP)

Evaluation Criteria: Code merge conflicts (in team environments) and Maintainability.

Achieved Results (Code Illustration): Complete separation of business validation logic and data persistence (storage).

Bad Code (Not Applied): One class carrying two distinct responsibilities.


```

1  public class StudentManager
3      public void Register(Student s)
4      {
5          // Responsibility 1: Business Logic Validation
6          if(s.Age < 18) throw new Exception("Invalid age");
7
8          // Responsibility 2: I/O Communication (File Writing)
9          File.AppendAllText("data.csv", $"{s.Id},{s.Name}");
10     }
11 }

```

Figure 32: Bad Code SRP

Good Code (Applied): Each class has only one reason to change.

```

0 references | 0 references
1  public class StudentValidator
2  {
3      0 references | 0 references
4      public bool IsValid(Student s) => s.Age >= 18;
5  }
6
7  0 references | 0 references
8  public class CsvStudentRepository
9  {
10     0 references | 0 references
11     public void Save(Student s) => File.AppendAllText("data.csv", $"{s.Id},{s.Name}");
12 }

```

Figure 33: Good Code SRP

Performance Conclusion: If not applied, a developer modifying the age validation logic could accidentally break the CSV file formatting, causing system-wide errors. Applying SRP makes the system safer and allows tasks to be easily divided among team members (one handles logic, another handles database) without stepping on each other's toes.

O - Open/Closed Principle (OCP)

Evaluation Criteria: Feature Extension Time and Regression Risk (breaking old features when adding new ones).

Achieved Results (Code Illustration): Adding a new data storage method (e.g., saving to a SQL database instead of CSV) without deleting or modifying existing code.

Bad Code (Not Applied): Forcing modifications on a perfectly working class.

```

1  ✓ public class StudentManager
2  {
3      // Every time a new storage method is added, this class must be modified
4      0 references | 0 references
5      public void SaveData(Student s, string databaseType)
6      {
7          if (databaseType == "CSV") { /* Write to CSV code */ }
8          else if (databaseType == "SQL") { /* SQL Server connection code */ }
9      }
10 }
```

Figure 34: Bad Code OCP

Good Code (Applied): Extending functionality via Interfaces.

```

1  public interface IStudentRepository { void Save(Student s); }
2  → Tab to Jump
3  // Old feature working perfectly (Untouched)
4  0 references | 0 references
5  public class CsvStudentRepository : IStudentRepository { ... }
6  // ADDING a new feature safely
7  0 references | 0 references
8  public class SqlStudentRepository : IStudentRepository
9  {
10     1 reference | 1 reference
11     public void Save(Student s) { /* SQL connection code */ }
12 }
```

Figure 35: Good Code OCP

Performance Conclusion: Ignoring OCP forces you to constantly modify core source code, creating "Regression Bugs". Applying OCP maximizes scaling performance: new features are plugged in as modules (plug-and-play), leaving existing, tested code strictly untouched.

L - Liskov Substitution Principle (LSP)

Evaluation Criteria: Runtime Reliability and Polymorphism Safety.

Achieved Results (Code Illustration): Ensuring all concrete classes strictly fulfill the contract required by the system, with no "unexpected" behaviors.

Bad Code (Not Applied): A child class breaking the parent's expected behavior.

```

1  public class BaseRepository
2  {
3      1 reference | 1 reference
4      public virtual void Delete(string id) { /* Deletion logic */ }
5  }
6
7      0 references | 0 references
8  public class ReadOnlyCsvRepository : BaseRepository
9  {
10     // Breaks logic: Calling this method will crash the application
11     1 reference | 1 reference
12     public override void Delete(string id)
13     {
14         throw new Exception("Error! This CSV is read-only, cannot delete!");
15     }
16 }

```

Figure 36: Bad Code LSP

Good Code (Applied): Re-categorizing hierarchy, never forcing a class to implement what it cannot do.

```

1  public interface IReadable { Student Get(string id); }
2      0 references | 0 references | 0 references | 0 references
3  public interface IWritable { void Delete(string id); }
4
5      // LSP respected: Anyone calling IReadable is safe
6      0 references | 0 references
7  public class ReadOnlyCsvRepository : IReadable
8  {
9      1 reference | 1 reference
10     public Student Get(string id) { /* Returns data */ }
11 }

```

Figure 37: Good Code LSP

Performance Conclusion: If LSP is violated, the program will compile successfully but unexpectedly crash during user interaction. Adhering to LSP provides stable execution performance and entirely eliminates the need for cumbersome type-checking blocks like `if (repository is ReadOnlyCsvRepository)`.

I - Interface Segregation Principle (ISP)

Evaluation Criteria: Dependency Payload and Compilation Time.

Achieved Results (Code Illustration): Preventing the Student Controller from being burdened with Faculty/Admin management methods.

Bad Code (Not Applied): A bloated "Fat Interface".

```

1  public interface IUniversityManager
2  {
3      1 reference | 1 reference
      void ViewGrades();    // Needed by Student
4      1 reference | 1 reference
      void DeleteCourse();  // Only needed by Admin
5  }
6
7  0 references | 0 references
  public class StudentApp : IUniversityManager
8  {
9      1 reference | 1 reference
      public void ViewGrades() { /* View grades logic */ }
10     // Forced to implement Admin method despite having no authorization
11     1 reference | 1 reference
      public void DeleteCourse() => throw new NotImplementedException();
12 }

```

Figure 38: Bad Code ISP

Good Code (Applied): Segregating interfaces based on roles.

```

1  1 reference | 1 reference | 1 reference | 1 reference
  public interface IStudentActions { void ViewGrades(); }
2  0 references | 0 references | 0 references | 0 references
  public interface IAdminActions { void DeleteCourse(); }
3
4  0 references | 0 references
  public class StudentApp : IStudentActions
5  {
6      1 reference | 1 reference
      public void ViewGrades() { /* Focuses strictly on Student tasks */ }
7  }

```

Figure 39: Good Code ISP

D - Dependency Inversion Principle (DIP)

Evaluation Criteria: Testability and Physical Coupling.

Achieved Results (Code Illustration): Inverting control. Business logic is no longer chained to a physical hard drive (where the CSV resides).

Bad Code (Not Applied): Hard-coding low-level objects.

```

1 public class StudentCourseManager
2 {
3     // Direct instantiation: Manager is physically tied to the CSV file
4     private CsvStudentRepository _repo = new CsvStudentRepository();
5
6     public void Enroll() { _repo.Save(...); }
7 }

```

Figure 40: Bad Code DIP

Good Code (Applied): Dependency Injection.

```

1 public class StudentCourseManager
2 {
3     private readonly IStudentRepository _repo;
4
5     // Depends on abstractions. Works with real CSV or Fake testing classes
6     public StudentCourseManager(IStudentRepository repo)
7     {
8         _repo = repo;
9     }
10
11     public void Enroll() { _repo.Save(...); }
12 }

```

Figure 41: Good Code DIP

Comprehensive SOLID Performance Comparison

SOLID Principle	Evaluation Criteria	Without Application (Bad Practice)	With Application (Good Practice)	Performance Conclusion & Justification
S - Single Responsibility	Code merge conflicts and Maintainability	One class carrying two distinct responsibilities (e.g., business logic validation and file writing).	Complete separation of business validation logic and data persistence (storage).	Makes the system safer and allows tasks to be easily divided among team members without stepping on each other's toes.
O - Open / Closed	Feature Extension Time and Regression Risk	Forcing modifications on a perfectly working class every time a	Extending functionality via Interfaces (plug-	Maximizes scaling performance; new features are added safely,

		new storage method is added.	and-play modules).	leaving existing, tested code strictly untouched.
L - Liskov Substitution	Runtime Reliability and Polymorphism Safety	A child class breaking the parent's expected behavior (e.g., throwing an exception when a method is called).	Re-categorizing hierarchy, never forcing a class to implement what it cannot do.	Provides stable execution performance and entirely eliminates the need for cumbersome type-checking blocks.
I - Interface Segregation	Dependency Payload and Compilation Time	A bloated "Fat Interface" forcing classes to implement methods they have no authorization for.	Segregating interfaces based on roles (e.g., separating student actions from admin actions).	Prevents controllers from being burdened with unrelated management methods, reducing dependency overhead.
D - Dependency Inversion	Testability and Physical Coupling	Direct instantiation (hard-coding low-level objects), tying the manager physically to the CSV file.	Dependency Injection, relying on abstractions rather than concrete implementations.	Inverts control, ensuring business logic is no longer chained to a physical hard drive, enabling automated unit testing.

Table 3: Comprehensive SOLID

III. Conclusion

In conclusion, the design and development of the Student Information Management System (SIMS) successfully demonstrates the profound impact of applying industry-standard software engineering practices. Throughout this report, we have systematically investigated the Object-Oriented Programming (OOP) paradigm and the critical role of SOLID development principles in creating robust, scalable applications. By integrating clean coding techniques, we ensured that the utilization of data structures and algorithms remains highly maintainable and efficient.

The architectural design phase, which leveraged ASP.NET Core to handle large CSV dataset processing, highlighted the practical application of these theoretical concepts. The strategic use of UML diagrams—specifically Use Case, Class, and Package diagrams—provided a clear and logical blueprint. Additionally, the systematic implementation of multiple design patterns across creational, structural, and behavioural categories effectively resolved complex architectural challenges. Furthermore, the formulation of a comprehensive testing regime, including provisions for automated testing, guarantees the system's ongoing reliability, security, and performance.

Ultimately, evaluating the SIMS application proves that adhering to rigorous design principles is not merely an academic exercise. It directly fulfills the university's modernization requirements for improved efficiency and maintainability, establishing a highly flexible and resilient foundation. Mastering these architectural concepts and quality attributes serves as a definitive stepping stone for building scalable, enterprise-level architectures in modern web environments and cross-platform mobile applications.